

</> Thread.start()

KajiJDK · Sistemas de bajo nivel

Concurrencia e hilos en una JVM en Rust

Informe técnico — qué hay y qué se podría hacer

Autor: **Esteban Nicolás Larena**

Fecha: **5 de agosto de 2026**

Contenido

1 · Resumen ejecutivo	3
2 · Estado actual (lo que ya corre)	3
3 · El rascacielos de la concurrencia	5
4 · Estado verificado contra el código	6
5 · Los dos cimientos: park/unpark y CAS	7
6 · Ruta crítica — GIL primero (GIL ≠ JIT)	7
7 · Plan por hitos (H0–H6)	8
8 · Consideraciones de Rust y arquitectura	10
9 · Riesgos y orden recomendado	11

1 · Resumen ejecutivo

Este informe es un **deep-dive del Hito A6 (concurrency)** de KajijDK: separa, contra el código real, **lo que ya funciona** de **lo que se podría hacer** para llegar a un *multithread profesional*. La meta no es el cómputo numérico del LLM —que vive en **kernels off-heap** (rayon/CUDA)— sino la **corrección** de la concurrency Java sobre el heap administrado.

El proyecto ya dio el **salto de sustrato (E1+E2)**: además de los **green threads cooperativos** (default y para el visor), hay un modo **hilos de SO reales + GIL** (`JVM_THREADS=os-gil`) donde cada `Thread.start()` lanza un `std::thread`, con `park / unpark` reales para monitores, `wait / notify` y `join`. Es la *planta baja* de un edificio cuya cumbre es `java.util.concurrent`.

*Tesis del informe (confirmada): el salto grande **no** era el JIT ni terminar el monitor, sino **cambiar el sustrato** de green threads a hilos de SO. Se hizo por el camino canónico — **GIL primero** (correcto pero sin paralelismo) y después **sacar el GIL** (E3), **ya hecho y ensanchado**: en `JVM_THREADS=os` corren en paralelo real el cómputo frame-local (lock-free), las lecturas compartidas (bajo `.read()`) y la asignación (lock-free, Eden en un arena Miri-verificado). El JIT es ortogonal (Fase F3) y no está en esta ruta.*

✓ **Estado:** E1+E2 + monitor completo + **API de Thread (H1) cerrada** — implementados y validados con **142 tests verdes, 0 warnings**. Las plantas 2 (monitor) y 3 (API de Thread) están completas: `interrupt()` despierta `sleep / join / wait` (con la carrera notify/interrupt resuelta por el GIL), y `getState()` reporta los seis estados. Y **el GIL ya se sacó** (E3/H3), y el paralelismo está **ensanchado**: en `JVM_THREADS=os`, (1) todos los opcodes **frame-local** (int/long/float/double, shifts, conversiones, refs, ramas) corren **lock-free**; (2) las ops compartidas de **solo-lectura** (`getField`, `arraylength`, array-loads) corren concurrentes bajo un `RwLock` (`.read()`); y (3) la **asignación** (`new / newarray`) es **lock-free**, con Eden en un arena `UnsafeCell` **verificado con Miri**. El `Arc<Mutex<JVM>>` de toda-la-VM desapareció; hay GC stop-the-world cooperativo. Y el **JMM (H4)** también está: **modelo relajado** —campos no-volatile lock-free (`Relaxed`), `volatile` con `Acquire / Release`, Eden sobre un sustrato atómico Miri-verificado— (detalle abajo). Y ya subimos a **CAS (H5)** — `compareAndSwap` intrínseco + `AtomicInteger / Long / Reference` — y al **núcleo de java.util.concurrent (H6): AQS** (exclusivo y compartido) con `ReentrantLock / Condition`, `Semaphore` y `CountDownLatch`, todo en Java y validado por el oráculo `green=os-gil=os`. Lo que queda es el **volumen de j.u.c** (pools, colecciones) y los locks finos.

Horizontes: **Base** el núcleo **Avanzado** segunda pasada **Cumbre** lo más difícil

2 · Estado actual (lo que ya corre)

Verificado en el intérprete (`bytecode_interpreter.rs` + los `invoke*`) y con tests verdes:

- **Green threads (Fase 0).** `GreenThread` = un call stack propio; scheduler **round-robin cooperativo** que conmuta en el *safe point* (entre opcodes); el stack activo vive en `self.frames` y se intercambia con el del hilo entrante. `Thread.start()` spawna (dispatch virtual de `run()`); `join` y `sleep` (reloj lógico de opcodes). El GC toma las raíces de **todos** los hilos (`parked()`). Verificado: dos workers + `main` → 100.
- **Monitores.** `Monitor` = dueño + conteo reentrante + `blocked-set` + `wait-set`. `synchronized` en sus dos formas: **bloques** (`monitorenter / monitorexit`) y **métodos** (`ACC_SYNCHRONIZED`, sin opcode — el VM toma/suelta el monitor del receptor, o del `Class` en `static synchronized`, en un único punto de *release* imposible de saltar, también en el *unwind* de excepción).
- **Señalización.** `wait / notify / notifyAll`: libera el monitor guardando el conteo de reentrada, parquea en el `wait-set` y lo re-adquiere al despertar. Verificado: exclusión mutua real ($2 \times 100 \rightarrow 200$ vs control sin lock que pierde updates) y handshake productor/consumidor (`wait` → `notify` → 42).
- `IllegalMonitorStateException`: gate `owns_monitor` sobre `monitorexit / wait / notify` (JVMS §6.5 / JLS 17.2).
- **Hilos de SO + GIL (E1+E2) ◀ nuevo.** Modo `JVM_THREADS=os-gil`: la VM vive tras un **GIL** (`Arc<Mutex<JVM>>`); cada `Thread.start()` lanza un `std::thread` que toma el lock por opcode y lo suelta. Bloqueo real con `park / unpark` (centralizado en `make_runnable`) para monitores, `wait / notify` y `join`; `Thread.sleep` en tiempo real. **El GC corre seguro bajo el GIL** (solo un hilo toca la VM) — el GIL es el stop-the-world por ahora.
- `wait(timeout)` ◀ **nuevo.** `Object.wait(long)`: espera temporizada que vuelve al vencer el plazo aunque nadie notifique. Deadline reusando `sleep_until`; al expirar, `expire_timed_block` saca al hilo del `wait-set` y la ruta de re-adquisición lo resume (un *self-notify*). Plazo por reloj de opcodes en green, **tiempo real** en OS. Verificado: `WaitTimeout` → 7 (green y OS).
- **Monitores GC-safe ◀ nuevo.** Los monitores se keyean por offset del heap; al mover objetos (minor y compact) las claves se **remapean por el forward del GC** (más `wait_reacquire` y el lock de método `synchronized`), y se descartan monitores de objetos colectados. **Ya se puede compactar con monitores tomados.** Verificado: `GcMonitor` (un `synchronized` + `System.gc()` que mueve el lock) → 5 (green y OS).
- **El seam.** El heap está tras `HeapService`, su único dueño; toda escritura de referencia pasa por `store_reference` (write barrier no-bypassable) — el punto donde entrará la sincronización fina cuando se **saque** el GIL (E3).

3 · El rascacielos de la concurrencia

Cada planta se apoya en la de abajo: no hay `ReentrantLock` sin CAS, ni CAS útil sin un modelo de memoria, ni modelo de memoria que *importe* sin hilos reales que lo hagan necesario.

RASCACIELOS DE LA CONCURRENCIA · estado tras H5 (CAS) + H6 (j.u.c)

[6] java.util.concurrent · lo que usa la gente ● HECHO

✓ AQS (excl+shared) · ReentrantLock · ReadWriteLock · Cond

✓ Semaphore · CountdownLatch · CyclicBarrier · ArrayBlockQ

✓ ThreadPoolExecutor · ScheduledExecutor · CompletableFut

✓ ConcurrentHashMap · Linked/Priority/Delay queues

deques · resize/lecturas lock-free CHM · locks finos

[5] Atómicos / CAS · raíz lock-free ✓ HECHO

✓ compareAndSwap · AtomicInteger / AtomicLong / Reference

[4] Modelo de Memoria (JMM) · relajado y fiel (H4) ✓ HECHO

✓ volatile=Acq/Rel · no-volatile Relaxed · no-tearing

[3] API de Thread · control de hilos ✓ HECHO

✓ start · run · join · sleep (sleep ya wall-clock en OS)

✓ interrupt · yield · currentThread · getState · isAlive

[2] Monitor intrínseco ● CASI

✓ monitorenter/exit · synchronized (bloque Y método)

✓ reentrancia · wait/notify/notifyAll · IMSE · park/unpark

✓ wait(timeout) · GC-safe (sobrevive compact/minor) ◀ NUEVO

✓ interrupts (H1) · ■ biased/thin locks · deadlock detect

[1] Substrato de ejecución · paralelismo real ✓ ● GRAN

✓ green threads (cooperativo, default + visor) AVANCE

✓ hilos de SO reales (std::thread/Thread.start) ◀ NUEVO (E1)

✓ GIL (Arc<Mutex<JVM>>, 1 opcode/lock) ◀ NUEVO (E1)

✓ park/unpark · GC seguro bajo el GIL (= STW gratis)

✓ GIL removido: frame-local lock-free en os (E3) ◀ NUEVO

✓ ensanchado (W1/W2/W3) · ✓ JMM (H4) · ■ locks finos

▲ el edificio en pie: quedan pools/colecciones y locks finos ▲

Estado: ✓ hecho ● casi ● parcial ■ falta

4 · Estado verificado contra el código

Planta por planta, lo que falta para «profesional» (revisado en `bytecode_interpreter.rs`, `invoke*.rs` y el `Thread.java` de `bootstrap/`):

Planta	Estado	Qué falta
6 · <code>java.util.concurrent</code>	✓ ● hecho (H6)	Hecho: AQS (<code>AbstractQueuedSynchronizer</code>) en modo exclusivo y compartido — base de casi todo —, <code>ReentrantLock</code> + <code>Condition</code> , <code>ReentrantReadWriteLock</code> , <code>Semaphore</code> , <code>CountDownLatch</code> , <code>CyclicBarrier</code> , <code>ArrayBlockingQueue</code> (productor/consumidor acotado), el framework Executor (<code>ThreadPoolExecutor</code> + <code>submit</code> / <code>Future</code> , y un <code>ScheduledThreadPoolExecutor</code> con min-heap por deadline), <code>ConcurrentHashMap</code> (encadenamiento + <i>lock striping</i>), <code>LinkedBlockingQueue</code> (dos locks), <code>PriorityBlockingQueue</code> (min-heap por prioridad), <code>DelayQueue</code> (elementos <code>Delayed</code> , espera hasta expirar) y <code>CompletableFuture</code> (<code>supplyAsync</code> / <code>thenApply</code> / <code>thenCompose</code> , más completación excepcional con <code>exceptionally</code> y <code>thenCombine</code> que fusiona dos futuros con un <code>BiFunction</code>), todos escritos <i>en Java</i> sobre <code>park</code> / <code>unpark</code> + CAS (o el monitor) y validados por el oráculo (<code>green≡os-gil≡os</code>). Falta: dequeues, <i>resize</i> /lecturas lock-free del <code>ConcurrentHashMap</code>
5 · Atómicos / CAS	✓ hecho (H5)	Hecho: <code>compareAndSwap</code> (la instrucción atómica raíz: <code>cas_u32</code> / <code>cas_u64</code> sobre el <code>AtomicRegion</code> , <code>AcqRel</code>) y <code>AtomicInteger</code> / <code>AtomicLong</code> / <code>AtomicReference</code> — la CAS de referencia pasa por el write barrier. Verificado (<code>AtomicLong</code> 64-bit + <code>AtomicReference</code> identity-CAS → 202)
4 · Modelo de Memoria (JMM)	✓ ● hecho (H4)	Modelo relajado: campos no-volatile lock-free (<code>Relaxed</code>), <code>volatile</code> = <code>Acquire</code> / <code>Release</code> , no-tearing de <code>long</code> / <code>double</code> (<code>AtomicU64</code>), Eden sobre un subtrato atómico Miri-verificado ; happens-before vía los átomos + el <code>RwLock</code> + monitores + <code>start</code> / <code>join</code> . Falta: semántica de <code>final</code> , <code>VarHandle</code>
3 · API de <code>Thread</code>	✓ hecho (H1)	Hecho: <code>start</code> / <code>run</code> / <code>join()</code> / <code>sleep(long)</code> , <code>yield</code> , <code>currentThread</code> , nombre/id, <code>isAlive</code> , <code>getState</code> (los seis estados de <code>Thread.State</code>), patrón <code>Runnable</code> (una lambda lo satisface), <code>start</code> -dos-veces → <code>IllegalThreadStateException</code> , e <code>interrupt</code> / <code>InterruptedException</code> que despierta <code>sleep</code> / <code>join</code> / <code>wait</code> (re-adquiere el monitor antes de lanzar en el <code>wait</code> ; la carrera <code>notify/interrupt</code> la resuelve el GIL). Periféricos hechos (2026-08): <code>ThreadLocal</code> (aislamiento por-hilo real, verificado con 4 hilos OS), prioridad (rango <code>[1,10]</code>), <code>daemon</code> (regla de ciclo de vida). Falta: <code>UncaughtExceptionHandler</code>
2 · Monitor intrínseco	● casi	Hecho: <code>monitorenter</code> / <code>exit</code> , <code>synchronized</code> (bloques y métodos, incl. <code>static</code>), reentrancia, <code>wait</code> / <code>notify</code> / <code>notifyAll</code> , IMSE, <code>park</code> / <code>unpark</code> real entre hilos de SO (E2), <code>wait(timeout)</code> , interrupción del wait (re-adquiere el monitor antes de lanzar la <code>InterruptedException</code> , JLS 17.2; la carrera <code>notify/interrupt</code> la resuelve el GIL — H1), y monitor GC-safe (las claves se remapean por el <code>forward</code> del GC en minor/compact → se puede compactar con monitores tomados). Falta: biased/thin locks, detección de deadlock
1 · Subtrato de ejecución	✓ ● gran avance	Hecho (E1+E2): green threads (cooperativo, default) y hilos de SO reales (<code>std::thread</code> por <code>Thread.start()</code>); <code>park</code> / <code>unpark</code> . Hecho (E3 — GIL removido): en <code>JVM_THREADS=os</code> los opcodes frame-local (aritmética int/stack/ramas) corren lock-free en paralelo sobre un <code>RunningCtx</code> por-hilo con code cache; solo <code>SharedVm</code> se lockea. GC que mueve objetos con safepoint STW cooperativo (<code>gc_pending</code> + <code>park</code> / <code>unpark</code>). <code>os-gil</code> queda como referencia serializada. Ensanchado: frame-local completo (W1), lecturas concurrentes bajo <code>RwLock</code> (W3), asignación lock-free con Eden en arena <code>UnsafeCell</code> Miri-verificado (W2). Falta: locks por-estructura finos, preempción

5 · Los dos cimientos: park/unpark y CAS

De todo el edificio, **dos primitivas** sostienen el resto: **park / unpark** (planta 1) y **CAS** (planta 5). De esas dos se deduce *casi todo* lo de arriba — AQS, los locks, los atómicos, los pools.

- **park / unpark** — bloquear/despertar un hilo sin *busy-wait*. Hoy el scheduler cede cada opcode (no hace falta), pero con hilos de SO es *la* primitiva de bloqueo. Sobre ella se construyen **LockSupport** → AQS → todos los locks.
- **CAS** (**compareAndSwap**) — la operación atómica raíz. Sin ella no hay estructuras lock-free ni AQS. En Rust se mapea directo a `std::sync::atomic` (`compare_exchange`).

6 · Ruta crítica — GIL primero (GIL ≠ JIT)

```

cerrar monitor      [E1+E2] ✓ HECHO      sacar el GIL ✓ HECHO      desbloquear "pro"
IMSE·wait·notify → OS threads + GIL → frame-local sin GIL → JMM ✓ → CAS ✓ → AQS ✓
(✓)                (✓ park/unpark)      (✓ E3 + ensanchado)  (✓ H4·H5·H6 núcleo)
                                     ▲ ACÁ: volumen j.u.c + locks finos

```

El camino canónico —el de CPython y las JVM tempranas— es **GIL primero**: hilos de SO con un *lock global* que serializa el intérprete (simple y correcto, pero todavía **sin paralelismo real**) y después **sacar el GIL** (locks finos por estructura + TLABs + GC stop-the-world). Recién ahí **volatile** y los *fences* dejan de ser decorativos: con green threads no hay reordenamientos reales que tapar; con hilos de SO, omitirlos rompe el código de forma no-determinista.

Aclaración importante (GIL ≠ JIT). El **GIL** es un lock que serializa el intérprete; habilita hilos de SO correctos sin reescribir la VM. El **JIT** compila bytecode a nativo y vive en la **Fase F3** (track *burst*), fuera de esta ruta. No se necesita JIT para tener hilos de SO ni para un JMM correcto: lo que cuesta es el **refactor de ownership**, no la generación de código nativo.

7 · Plan por hitos (H0–H6)

Cada hito con su criterio de éxito medible. **H2 (el salto grande) ya está hecho**; H0 quedó cerrado salvo la interrupción del `wait`, que vive en H1 — el próximo hito pendiente.

H0 Cerrar el monitor intrínseco Avanzado · ✓ cerrado (interrupt-of-wait incluido, con H1)

Base ✓ `IllegalMonitorStateException` (gate owns_monitor) — hecho

Avanzado ✓ `Object.wait(long)` con timeout — deadline sobre `sleep_until`; real-time en OS (demo `WaitTimeout` → 7)

Cumbre ✓ Monitor GC-safe — las claves se remapean por el `forward` del GC en minor/compact (+ `wait_reacquire` y el lock de método); demo `GcMonitor` → 5

Avanzado ✓ Interrupción del `wait` → `InterruptedException` (parte de los interrupts, H1)

✓ **Logrado:** `wait(t)` vence por tiempo y el monitor sobrevive a un GC que mueve su objeto (compact/minor) sin perder dueño/wait-set — verificado (y el test se cuelga si se quita el remapeo).

H1 API de Thread completa Avanzado · ✓ HECHO

Base ✓ `currentThread()`, `yield()`, nombre/id, `isAlive()`, `getState()` (los seis estados; `NEW` derivado de la ausencia de slot)

Avanzado ✓ `interrupt()` / `isInterrupted()` / `interrupted()` + `InterruptedException` que despierta `sleep` / `join` / `wait`. La bandera vive en el objeto (observable aun en `NEW`); el throw se lanza desde el pc de la llamada y re-adquiere el monitor antes en el caso de `wait` (JLS 17.2)

Avanzado ✓ Patrón `new Thread(runnable)` (interfaz `Runnable` + campo `target`; una lambda lo satisface); `start` dos veces → `IllegalThreadStateException`

Avanzado ✓ Periféricos (2026-08): `ThreadLocal` (lista de asociación por-hilo, identidad GC-safe, prioridad ([1,10]), daemon (atributo + ciclo de vida) · `UncaughtExceptionHandler`

✓ **Logrado:** `new Thread(() -> ...).start()` corre la lambda; `t.interrupt()` saca a un hilo de `sleep` / `join` / `wait` con `InterruptedException` (verificado `Thread.holdsLock` en el catch del `wait`); `getState()` reporta fiel. La máquina de estados quedó consolidada en un punto único de bloqueo — justo lo que el handshake de safeport de H3 va a leer.

H2 Substrato: hilos de SO + GIL Cumbre · ✓ HECHO (E1+E2)

Cumbre Cada `GreenThread` → un `std::thread` real (carrier propio)

Cumbre **GIL:** un lock global del intérprete tomado entre opcodes y cedido en cada safeport (correcto, todavía serializado)

Cumbre `park` / `unpark` reales sobre el SO (base de `wait` / `join` / contención)

Cumbre Refactor de ownership: heap/monitors/metaspaces compartidos tras el seam `HeapService.store_reference` (`Arc` + `Send` / `Sync`)

✓ **Logrado:** N hilos de SO ejecutan el mismo programa serializados por el GIL, sin *data races* — validado por 4 tests de modo OS (mismo resultado que en green), 127 verdes en total.

H3 Sacar el GIL — paralelismo real Cumbre · ✓ HECHO + ensanchado

Cumbre ✓ Estado de ejecución **por-hilo:** cada hilo de SO posee su `RunningCtx` (pila + pc + **code cache**); solo `SharedVm` vive tras un lock. El `Arc<Mutex<JVM>>` de toda-la-VM desapareció

Cumbre ✓ **Fast-path lock-free** (`run_frame_local`): aritmética int, stack shuffles y ramas int corren sin lock; el resto cae al `run_one` lockeado. Clasificación conservadora (un error solo quita paralelismo, nunca corre un op compartido sin lock)

Cumbre ✓ GC **stop-the-world cooperativo** (`coordinate_gc`): `gc_pending` atómico + `park` / `unpark`; todos sincronizan su frame al slot y paran antes de mover objetos, después recargan los frames remapeados

Cumbre ✓ **Ensanchado:** (W1) frame-local completo — int/long/float/double, shifts, conversiones, refs, ramas; (W3) lecturas compartidas concurrentes bajo `RwLock` — `getfield` / `arraylength` / `array-loads`; (W2) **asignación lock-free** — `new` / `newarray`, Eden en un arena `UnsafeCell` **Miri-verificado**. ✓ Falta: ensanchar más (resolución de métodos) y locks por-estructura finos (el JMM, H4, ya está — abajo)

✓ **Logrado:** los opcodes frame-local escalan en paralelo real sin romper invariantes del GC. Validado por el oráculo (`os` ≡ `os-gil` ≡ green) + **stress test** (3 hilos, cómputo lock-free + GC handshake). El stress test es *señal, no prueba* de ausencia de races.

H4 Modelo de memoria (JMM) — relajado y fiel Cumbre · ✓ HECHO

Cumbre ✓ **El hallazgo:** tras W1/W2/W3 todo acceso a campos pasaba por el `RwLock` → la VM sobre-sincronizaba (SC), así que el JMM ya se cumplía. H4 no fue *agregar* sincronización sino **relajar**, como HotSpot: campos no-volatile lock-free, `volatile` repone las garantías donde hacen falta

Cumbre ✓ **Substrato atómico** (`AtomicRegion`, celdas `UnsafeCell<u64>` 8-alineadas, **Miri-verificado**): Eden entero se apoya en él. Campos **no-volatile** → `Relaxed` (lock-free, racy pero *sound*); `volatile` → `Acquire` / `Release`. En Rust "relajar" ≠ bytes planos concurrentes (eso es UB) — la forma *sound* es con átomos

Cumbre ✓ **No-tearing** de `long` / `double`: una sola regla de layout (`place_field`) 8-alinea los category-2 → `volatile long` por un `AtomicU64` real. Happens-before vía los átomos + el `RwLock` + monitores + `start` / `join`

Cumbre ✓ `getfield` / `putfield` relajados: `putfield` primitivo a objetos de Eden es **lock-free** (fuera del `.write()`); `volatile` usa el ordering fiel. Los objetos jóvenes son el hot-path

✓ **Éxito:** test de **publicación volatile** (publisher escribe payload + flag; reader spinnea el flag y ve el payload, nunca el default) verde en `green=os-gil=os`; no-tearing de un `long volatile`. La soundness del núcleo `unsafe` es **Miri**. (Nota: *relajar* hizo del *cache de condy* un *data-race latente* — se resolvió haciéndolo **raíz de GC**, remapeado por el *forward* del colector.)

H5 Atómicos / CAS Cumbre · ✓ **HECHO**

Cumbre ✓ `compareAndSwap` intrínseco → `cas_u32` / `cas_u64` sobre el `AtomicRegion` (`compare_exchange` `AcqRel` / `Acquire`), la CAS de referencia por el **write barrier**

Cumbre ✓ `AtomicInteger` / `AtomicLong` / `AtomicReference` (`getAndAdd` , `getAndSet` , `compareAndSet`), escritos *en Java* sobre el intrínseco

✓ **Éxito:** un contador con CAS en bucle desde varios hilos da el total exacto, sin lock — verificado (`AtomicLong` 64-bit + `AtomicReference` identity-CAS → 202), `green=0s-gil=0s` .

H6 `java.util.concurrent` Cumbre · ✓ ● **HECHO (núcleo AQS)**

Cumbre ✓ **AQS** (`AbstractQueuedSynchronizer`) sobre `park` / `unpark` + CAS, en **modo exclusivo y compartido** (cola de espera Treiber + wake-all) — la base de casi todo

Cumbre ✓ `ReentrantLock` + `Condition` y `ReentrantReadWriteLock` (exclusivo/compartido), `Semaphore` , `CountDownLatch` y `CyclicBarrier` — todo *en Java* sobre AQS (o el monitor)

Cumbre ✓ `ArrayBlockingQueue` — cola acotada FIFO (productor/consumidor) sobre `ReentrantLock` + 2 `Condition` ; la primera colección de `j.u.c`

Cumbre ✓ **Framework Executor** : `ThreadPoolExecutor` de tamaño fijo (shutdown por *poison-pill*) + `submit` / `Future` / `FutureTask` / `Callable` , y un `ScheduledThreadPoolExecutor` (delay/periódico sobre un min-heap por deadline; motivó el intrínseco `System.nanoTime`)

Cumbre ✓ **Colecciones/futuros**: `ConcurrentHashMap` (encadenamiento + *lock striping*; motivó `Object.equals`), `LinkedBlockingQueue` (dos locks), `PriorityBlockingQueue` (min-heap, orden por `Comparator` / `Comparable`), `DelayQueue` (elementos `Delayed` , espera temporizada hasta expirar; usó `System.nanoTime`) y `CompletableFuture` (`supplyAsync` / `thenApply` / `thenCompose` / `exceptionally` / `thenCombine` ; motivó `java.util.function` + `Objects`)

Cumbre ■ deque (`ArrayDeque` / `ConcurrentLinkedDeque`) y *resize*/lecturas lock-free del `ConcurrentHashMap` (volumen restante, todo se deduce de AQS)

✓ **Éxito:** el núcleo de sincronizadores **y las primeras utilidades** corren en tu JVM —

`ReentrantLock` / `ReadWriteLock` / `Semaphore` / `CountDownLatch` / `CyclicBarrier` sobre un único AQS y el productor/consumidor de `ArrayBlockingQueue` , con `green=0s-gil=0s` . Falta el volumen de colecciones/pools.

8 · Consideraciones de Rust y arquitectura

El costo real del salto de substrato **no es el JIT**: es el **refactor de ownership**. Hoy toda la VM es `&mut self` con un único dueño (heap, monitores, metaspaces, frames). Hilos de SO = estado compartido y sincronizado.

- **El seam ya está puesto.** `HeapService.store_reference` es el portón no-bypasable por donde pasa toda escritura de referencia (hoy aplica el write barrier). Es exactamente donde entrará el lock/atomicidad cuando los hilos sean de SO — no hay que buscar todos los sitios de escritura.
- **Send / Sync.** El estado compartido vive tras `Arc<Mutex<...>>` (o locks por estructura en H3). El **GIL es el primer paso correcto**: un solo `Mutex` grande, fácil de razonar, antes de afinar.
- **El GIL se cede en safepoints.** Ya existe el safepoint entre opcodes (lo usa el GC). Es el mismo punto donde el hilo suelta el GIL y donde el GC stop-the-world hará su *handshake* en H3.
- **Differential / stress testing como oráculo.** El intérprete green-thread es la referencia de corrección: el mismo programa por los dos substratos debe dar el mismo resultado; los tests de estrés (carreras conocidas) cazan los *data races* al sacar el GIL.
- **El cómputo del LLM no entra acá.** El paralelismo pesado vive en kernels off-heap (rayon/CUDA); esta torre es para la *corrección* de la concurrencia Java sobre el heap, no para el número.

9 · Riesgos y orden recomendado

Hito	Riesgo / costo	Nota
H0 · cerrar monitor	Bajo	✅ cerrado — IMSE, <code>wait(timeout)</code> y monitor GC-safe hechos; solo resta la interrupción del <code>wait</code> , que se hace con H1
H1 · API Thread	Bajo–medio	✅ HECHO . El cableado + <code>getState()</code> + <code>interrupt()</code> (despierta sleep/join/wait); la carrera notify/interrupt la resuelve el GIL keyeando por estado. Periféricos hechos (2026-08): <code>ThreadLocal</code> , prioridad, daemon. Falta <code>UncaughtExceptionHandler</code>
H2 · OS threads + GIL	Alto	✅ HECHO (E1+E2) — refactor a <code>Arc<Mutex<JVM>></code> + <code>std::thread</code> ; GIL = correcto pero serializado. 142 tests verdes
H3 · sacar el GIL	Alto	✅ HECHO + ensanchado . Cómputo frame-local lock-free (<code>run_frame_local</code>), lecturas concurrentes bajo <code>RwLock</code> (<code>run_read_shared</code>) y asignación lock-free (Eden en arena <code>UnsafeCell</code>), con safepoint STW cooperativo (<code>coordinate_gc</code>); <code>os-gil</code> /green como oráculos. Los <i>data races</i> se cazan con el stress test y Miri (el <code>unsafe</code> está probado sound). Falta: locks por-estructura finos
H4 · JMM	Alto	✅ HECHO — modelo relajado . Substrato atómico (<code>AtomicRegion</code> , Miri-verificado); no-volatile <code>Relaxed</code> lock-free, <code>volatile</code> <code>Acquire</code> / <code>Release</code> , no-tearing (<code>AtomicU64</code>), <code>place_field</code> 8-alinea los category-2. El cache de condy se hizo raíz de GC. Test de publicación <code>volatile</code> <code>green=os-gil=os</code>
H5 · CAS	Medio	✅ HECHO . <code>cas_u32</code> / <code>cas_u64</code> sobre el <code>AtomicRegion</code> (mapea directo a <code>std::sync::atomic::compare_exchange</code>); <code>AtomicInteger</code> / <code>Long</code> / <code>Reference</code> en Java, la CAS de referencia por el write barrier
H6 · j.u.c	Alto (volumen)	✅ 🟡 núcleo + primeras utilidades HECHO . AQS (exclusivo + compartido) con <code>ReentrantLock</code> / <code>Condition</code> , <code>ReentrantReadWriteLock</code> , <code>Semaphore</code> , <code>CountDownLatch</code> , <code>CyclicBarrier</code> , <code>ArrayBlockingQueue</code> / <code>LinkedBlockingQueue</code> / <code>PriorityBlockingQueue</code> / <code>DelayQueue</code> , el framework <code>Executor</code> (<code>ThreadPoolExecutor</code> + <code>ScheduledThreadPoolExecutor</code>), <code>ConcurrentHashMap</code> y <code>CompletableFuture</code> (incl. <code>exceptionally</code> / <code>thenCombine</code>) — casi todo se deduce de AQS = park/unpark + CAS. Falta el volumen: dequeues, <i>resize</i> /lecturas lock-free del <code>ConcurrentHashMap</code>

Orden: H0 ✅ (monitor cerrado) → H2 ✅ (substrato) → H1 ✅ (API de `Thread` + `interrupt`) → H3 ✅ (sacar el GIL — paralelismo real, ensanchado (W1/W2/W3)) → H4 ✅ (JMM relajado — `volatile`/átomos, ya con paralelismo real) → H5 ✅ (CAS — átomos) → H6 ✅ 🟡 (`java.util.concurrent` — núcleo AQS + primeras utilidades:

`ReentrantLock` / `ReadWriteLock` / `Semaphore` / `CountDownLatch` / `CyclicBarrier` / `ArrayBlockingQueue` / `LinkedBlockingQueue` / `PriorityBlockingQueue` / `DelayQueue` / —incl. `exceptionally` / `thenCombine` —; falta solo dequeues + *resize*/lecturas lock-free del CHM).

Una sola frase: el JIT no habilita los hilos de SO; el refactor de ownership sí — y ya está hecho, incluido **sacar el GIL** y ensancharlo: en `JVM_THREADS=os` el cómputo, las lecturas y la asignación corren en paralelo real (con el `unsafe` Miri-verificado). El **JMM (H4) relajado**, la **CAS (H5)** y el **núcleo de `java.util.concurrent` (H6, sobre AQS)** ya están; lo que queda es el **volumen de j.u.c** (pools, colecciones) y afinar los locks finos.

✓ **Siguiente paso concreto:** con H4 (JMM), H5 (CAS) y el **núcleo de H6 (AQS** exclusivo+compartido → `ReentrantLock` / `Condition`, `Semaphore`, `CountDownLatch`) ya hechos y validados por el oráculo, lo que sigue es el **volumen restante de `java.util.concurrent`** (`BlockingQueue`, `ConcurrentHashMap`, `Executor` /pools) y los **locks por-estructura finos**. Nota: los green threads y `os-gil` no se retiran — quedan como **oráculo determinista** de corrección (mismo `.class`, tres substratos, mismo resultado).

⚠ **Honestidad (verificado contra el código):** el modo `JVM_THREADS=os` (paralelismo real) tiene un **bug residual conocido** — una referencia *stale* puede alcanzar un frame bajo **GC intenso concurrente** (repro `gc_race_stress`), disparando un receptor corrupto. Se acotó y arregló un mecanismo (el guard `had_local_frames` en el safepoint, que eliminó las raíces perdidas por un doble-swap), pero queda un **segundo mecanismo abierto**: un data race de bajo nivel en la *integración* del intérprete lock-free (todos los invariantes lógicos se cumplen; el remap del GC es demostrablemente correcto), cuyo próximo paso es un **ThreadSanitizer** sobre el loop real. **Los modos `green` y `os-gil` (el oráculo) no están afectados** — `os` está documentado como experimental.